

❓1. Write a program to find the greatest of n given numbers.

Aim

To write a C program that accepts n numbers from the user and finds the greatest (maximum) among them.

Algorithm

1. Start the program.
2. Input the value of n (number of elements).
3. Read n numbers one by one.
4. Initialize a variable max with the first number.
5. Compare each subsequent number with max:
 - If the number is greater than max, update max.
6. After all comparisons, max will hold the greatest number.
7. Display the result.
8. End the program.

code

```
#include <stdio.h>

int main() {
    int n, i;
    float num, max;

    printf("Enter how many numbers: ");
    scanf("%d", &n);

    printf("Enter number 1: ");
    scanf("%f", &num);
    max = num; // initialize max with first number
```

```

for(i = 2; i <= n; i++) {
    printf("Enter number %d: ", i);
    scanf("%f", &num);

    if(num > max) {
        max = num; // update max if current number is greater
    }
}

printf("\nGreatest number is: %.2f\n", max);

return 0;
}

```

output

Enter how many numbers: 5

Enter number 1: 12

Enter number 2: 45

Enter number 3: 7

Enter number 4: 89

Enter number 5: 23

Greatest number is: 89.00

2. Write a program to find the smallest of n given numbers.

Aim

To write a C program that accepts n numbers from the user and finds the smallest (minimum) among them.

Algorithm

1. Start the program.
2. Input the value of n (number of elements).
3. Read the first number and assign it to min.
4. For each of the remaining numbers:
 - Compare with min.
 - If the number is smaller than min, update min.
5. After all comparisons, min will hold the smallest number.
6. Display the result.
7. End the program.

Code

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i;
```

```
    float num, min;
```

```
    printf("Enter how many numbers: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number 1: ");
```

```
    scanf("%f", &num);
```

```
    min = num; // initialize min with first number
```

```
    for(i = 2; i <= n; i++) {
```

```
        printf("Enter number %d: ", i);
```

```
        scanf("%f", &num);
```

```
        if(num < min) {
```

```
        min = num; // update min if current number is smaller
    }
}

printf("\nSmallest number is: %.2f\n", min);

return 0;
}
```

Output

Enter how many numbers: 5

Enter number 1: 12

Enter number 2: 45

Enter number 3: 7

Enter number 4: 89

Enter number 5: 23

Smallest number is: 7.00

3. Write a program to print first 50 prime numbers.

Aim

To write a C program that prints the first 50 prime numbers

Algorithm

1. Start the program.
2. Initialize a counter to keep track of how many prime numbers have been printed.
3. Start checking numbers from 2 onwards (since 2 is the first prime).
4. For each number:

- Check if it is prime by testing divisibility from 2 up to its square root.
 - If it is prime, print it and increment the counter.
5. Stop when 50 prime numbers have been printed.
 6. End the program.

Code

```
#include <stdio.h>

#include <math.h>

int main() {

    int count = 0;    // to count prime numbers
    int num = 2;      // starting number
    int i, isPrime;

    printf("First 50 prime numbers are:\n");

    while(count < 50) {
        isPrime = 1; // assume number is prime

        // check divisibility up to sqrt(num)
        for(i = 2; i <= sqrt(num); i++) {
            if(num % i == 0) {
                isPrime = 0; // not prime
                break;
            }
        }
```

```

    }

    if(isPrime) {
        printf("%d ", num);
        count++;
    }

    num++;
}

printf("\n");
return 0;
}

```

Output

First 50 prime numbers are:

```

2 3 5 7 11 13 17 19 23 29
31 37 41 43 47 53 59 61 67 71
73 79 83 89 97 101 103 107 109 113
127 131 137 139 149 151 157 163 167 173
179 181 191 193 197 199 211 223 227 229

```

4. Write a program to reverse the digits of a given number

Aim

To write a C program that takes a number as input and prints its digits in reverse order.

Algorithm

1. Start the program.
2. Input a number from the user.
3. Initialize a variable $rev = 0$ to store the reversed number.
4. Repeat until the number becomes 0:
 - Extract the last digit using $digit = num \% 10$.
 - Update $rev = rev * 10 + digit$.
 - Remove the last digit from num using $num = num / 10$.
5. Print the reversed number.
6. End the program.

Code

```
#include <stdio.h>
```

```
int main() {
```

```
    int num, rev = 0, digit;
```

```
    printf("Enter a number: ");
```

```
    scanf("%d", &num);
```

```
    while(num != 0) {
```

```
        digit = num % 10;    // extract last digit
```

```
        rev = rev * 10 + digit; // build reversed number
```

```
        num = num / 10;      // remove last digit
```

```
    }
```

```
printf("Reversed number is: %d\n", rev);

return 0;

}
```

Output

Enter a number: 12345

Reversed number is: 54321

5 Write a program to calculate the sum of digits of the given number.

Aim

To write a C program that calculates the sum of digits of a given number.

Algorithm

1. Start the program.
2. Input a number from the user.
3. Initialize a variable sum = 0 to store the sum of digits.
4. Repeat until the number becomes 0:
 - Extract the last digit using digit = num % 10.
 - Add the digit to sum.
 - Remove the last digit from num using num = num / 10.
5. Print the final value of sum.
6. End the program.

Code

```
#include <stdio.h>
```

```
int main() {
```



```
int num, digit, sum = 0;

printf("Enter a number: ");
scanf("%d", &num);

while(num != 0) {
    digit = num % 10;    // extract last digit
    sum = sum + digit;   // add digit to sum
    num = num / 10;     // remove last digit
}

printf("Sum of digits = %d\n", sum);

return 0;
}
```

Output

Enter a number: 12345

Sum of digits = 15

6 Write a program to print Fibonacci series up to n terms.

Aim

To write a C program that prints the Fibonacci series up to n terms.

Algorithm

1. Start the program.
2. Input the number of terms n.

3. Initialize the first two terms: $a = 0$, $b = 1$.
4. Print the first two terms.
5. Use a loop to generate the next terms:
 - $next = a + b$
 - Print next
 - Update values: $a = b$, $b = next$
6. Continue until n terms are printed.
7. End the program.

Code

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i;
```

```
    long long a = 0, b = 1, next;
```

```
    printf("Enter the number of terms: ");
```

```
    scanf("%d", &n);
```

```
    printf("Fibonacci series up to %d terms:\n", n);
```

```
    for(i = 1; i <= n; i++) {
```

```
        printf("%lld ", a);
```

```
        next = a + b;
```

```
        a = b;
```

```
        b = next;
```

```
}

printf("\n");

return 0;

}
```

Output

Enter the number of terms: 10

Fibonacci series up to 10 terms:

0 1 1 2 3 5 8 13 21 34

7 Write a program to perform linear search on an array.

Aim

To write a C program that performs **linear search** on an array to find the position of a given element.

Algorithm

1. Start the program.
2. Input the size of the array (n).
3. Read n elements into the array.
4. Input the element to be searched (key).
5. Traverse the array from the first element to the last:
 - If `arr[i] == key`, print the position and stop.
6. If the element is not found after traversing the entire array, display a message saying "Element not found".
7. End the program.

Code

```
#include <stdio.h>
```

```
int main() {  
    int n, i, key, arr[100];  
  
    int found = 0;  
  
    printf("Enter the size of the array: ");  
    scanf("%d", &n);  
  
    printf("Enter %d elements:\n", n);  
    for(i = 0; i < n; i++) {  
        scanf("%d", &arr[i]);  
    }  
  
    printf("Enter the element to search: ");  
    scanf("%d", &key);  
  
    for(i = 0; i < n; i++) {  
        if(arr[i] == key) {  
            printf("Element %d found at position %d\n", key, i+1);  
            found = 1;  
            break;  
        }  
    }  
  
    if(!found) {
```

```
        printf("Element %d not found in the array\n", key);
    }

    return 0;
}
```

Output

Enter the size of the array: 5

Enter 5 elements:

10 25 30 45 50

Enter the element to search: 30

Element 30 found at position 3

8 Write a program to perform Binary search.

Aim

To write a C program that performs **binary search** on a sorted array to find the position of a given element.

Algorithm

1. Start the program.
2. Input the size of the array (n).
3. Read n elements into the array (ensure they are sorted).
4. Input the element to be searched (key).
5. Initialize two variables: low = 0, high = n-1.
6. Repeat until low <= high:
 - Find the middle index: $\text{mid} = (\text{low} + \text{high}) / 2$.
 - If $\text{arr}[\text{mid}] == \text{key}$, element is found → print position and stop.

- If $\text{arr}[\text{mid}] < \text{key}$, search the right half $\rightarrow$ set $\text{low} = \text{mid} + 1$.
 - If $\text{arr}[\text{mid}] > \text{key}$, search the left half $\rightarrow$ set $\text{high} = \text{mid} - 1$.
7. If the loop ends without finding the element, print "Element not found".
 8. End the program.

Code

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, key, arr[100];
```

```
    int low, high, mid, found = 0;
```

```
    printf("Enter the size of the array: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter %d sorted elements:\n", n);
```

```
    for(i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    printf("Enter the element to search: ");
```

```
    scanf("%d", &key);
```

```
    low = 0;
```

```
    high = n - 1;
```

```

while(low <= high) {
    mid = (low + high) / 2;

    if(arr[mid] == key) {
        printf("Element %d found at position %d\n", key, mid + 1);
        found = 1;
        break;
    }
    else if(arr[mid] < key) {
        low = mid + 1; // search right half
    }
    else {
        high = mid - 1; // search left half
    }
}

if(!found) {
    printf("Element %d not found in the array\n", key);
}

return 0;
}

```

Output

Enter the size of the array: 6

Enter 6 sorted elements:

10 20 30 40 50 60

Enter the element to search: 40

Element 40 found at position 4

9 Write a program to implement hashing using the division method with: (i) Linear probing (ii) Quadratic probing

Display the hash table and number of collisions.

Aim

To write a C program that implements hashing using the **division method** and resolves collisions using:

1. **Linear probing**
2. **Quadratic probing**

The program should display the hash table and the number of collisions.

Algorithm

Division Method

- Hash function:

$$h(k) = k \% m$$

- where k = key, m = table size.

Linear Probing

1. Compute hash index using division method.
2. If the slot is empty, insert the key.
3. If occupied, move sequentially ($\text{index} = (\text{index} + 1) \% m$) until an empty slot is found.
4. Count collisions whenever a slot is already occupied.

Quadratic Probing

1. Compute hash index using division method.

2. If the slot is empty, insert the key.
3. If occupied, probe using quadratic increments:

$\text{index} = (h(k) + i^2) \% m$

4. where $i = 1, 2, 3, \dots$
5. Count collisions whenever a slot is already occupied.

Code

```
#include <stdio.h>

#define SIZE 10 // hash table size

void linearProbing(int keys[], int n) {
    int hashTable[SIZE];
    int i, j, index, collisions = 0;

    for(i = 0; i < SIZE; i++)
        hashTable[i] = -1;

    for(i = 0; i < n; i++) {
        index = keys[i] % SIZE;

        if(hashTable[index] == -1) {
            hashTable[index] = keys[i];
        } else {
            collisions++;
            j = (index + 1) % SIZE;
            while(hashTable[j] != -1) {
```

```

        collisions++;
        j = (j + 1) % SIZE;
    }
    hashTable[j] = keys[i];
}
}

```

```

printf("\nLinear Probing Hash Table:\n");
for(i = 0; i < SIZE; i++)
    printf("%d : %d\n", i, hashTable[i]);
printf("Number of collisions: %d\n", collisions);
}

```

```

void quadraticProbing(int keys[], int n) {
    int hashTable[SIZE];
    int i, j, index, k, collisions = 0;

    for(i = 0; i < SIZE; i++)
        hashTable[i] = -1;

    for(i = 0; i < n; i++) {
        index = keys[i] % SIZE;

        if(hashTable[index] == -1) {
            hashTable[index] = keys[i];

```

```

    } else {
        collisions++;
        for(k = 1; k < SIZE; k++) {
            j = (index + k*k) % SIZE;
            if(hashTable[j] == -1) {
                hashCode[j] = keys[i];
                break;
            } else {
                collisions++;
            }
        }
    }
}

printf("\nQuadratic Probing Hash Table:\n");
for(i = 0; i < SIZE; i++)
    printf("%d : %d\n", i, hashCode[i]);
printf("Number of collisions: %d\n", collisions);
}

```

```

int main() {
    int keys[] = {23, 43, 13, 27, 56, 76, 89};
    int n = sizeof(keys)/sizeof(keys[0]);

    linearProbing(keys, n);
}

```

```
    quadraticProbing(keys, n);  
  
    return 0;  
}
```

Output

Linear Probing Hash Table:

0 : -1

1 : -1

2 : -1

3 : 23

4 : 43

5 : -1

6 : 56

7 : 27

8 : 76

9 : 89

Number of collisions: 3

Quadratic Probing Hash Table:

0 : -1

1 : -1

2 : -1

3 : 23

4 : 43

5 : -1

6 : 56

7 : 27

8 : 76

9 : 89

Number of collisions: 2

10 Write a program to implement a linked list.

Aim

To write a C program that implements a **singly linked list** with basic operations such as insertion and display.

Algorithm

1. Define a struct Node with two parts:
 - data (to store the value)
 - next (pointer to the next node).
2. Create a function to insert a new node at the end of the list.
3. Create a function to display all nodes in the list.
4. In main():
 - Initialize the list as empty (head = NULL).
 - Insert a few nodes.
 - Display the list.
5. End the program.

Code

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Define node structure
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* next;
```

```
};
```

```
// Function to insert node at the end
```

```
void insertEnd(struct Node** head, int value) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    newNode->data = value;
```

```
    newNode->next = NULL;
```

```
    if(*head == NULL) {
```

```
        *head = newNode;
```

```
        return;
```

```
    }
```

```
    struct Node* temp = *head;
```

```
    while(temp->next != NULL) {
```

```
        temp = temp->next;
```

```
    }
```

```
    temp->next = newNode;
```

```
}
```

```
// Function to display linked list
```

```
void display(struct Node* head) {  
    struct Node* temp = head;  
    printf("Linked List: ");  
    while(temp != NULL) {  
        printf("%d -> ", temp->data);  
        temp = temp->next;  
    }  
    printf("NULL\n");  
}  
  
int main() {  
    struct Node* head = NULL; // empty list  
  
    // Insert elements  
    insertEnd(&head, 10);  
    insertEnd(&head, 20);  
    insertEnd(&head, 30);  
    insertEnd(&head, 40);  
  
    // Display list  
    display(head);  
  
    return 0;  
}
```

Output

Linked List: 10 -> 20 -> 30 -> 40 -> NULL

11  Write a program to implement insertion in a linked list for the following three cases: (i) Inserting a new node at the beginning (ii) Inserting a new node at the end (iii) Inserting a new node at a specific position in the list

Aim

To write a C program that implements insertion in a **singly linked list** for three cases:

1. Inserting a new node at the beginning
2. Inserting a new node at the end
3. Inserting a new node at a specific position in the list

Algorithm

1. Define a struct Node with two fields:
 - data (to store the value)
 - next (pointer to the next node).
2. Create functions for insertion:
 - **Insert at beginning:**
 - Create a new node.
 - Point its next to the current head.
 - Update head to the new node.
 - **Insert at end:**
 - Traverse to the last node.
 - Point last node's next to the new node.
 - **Insert at specific position:**
 - Traverse to the node just before the desired position.
 - Insert the new node by adjusting pointers.

3. Create a function to display the linked list.
4. In main(), demonstrate all three insertion cases.

Code

```
#include <stdio.h>

#include <stdlib.h>

// Define node structure
struct Node {
    int data;
    struct Node* next;
};

// Function to display linked list
void display(struct Node* head) {
    struct Node* temp = head;
    printf("Linked List: ");
    while(temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Insert at beginning
void insertBeginning(struct Node** head, int value) {
```

```
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
newNode->data = value;  
newNode->next = *head;  
*head = newNode;  
}
```

// Insert at end

```
void insertEnd(struct Node** head, int value) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = value;  
    newNode->next = NULL;  
  
    if(*head == NULL) {  
        *head = newNode;  
        return;  
    }  
}
```

```
struct Node* temp = *head;  
while(temp->next != NULL) {  
    temp = temp->next;  
}  
temp->next = newNode;  
}
```

// Insert at specific position

```
void insertAtPosition(struct Node** head, int value, int pos) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = value;  
  
    if(pos == 1) { // insert at beginning  
        newNode->next = *head;  
        *head = newNode;  
        return;  
    }  
  
    struct Node* temp = *head;  
    for(int i = 1; i < pos-1 && temp != NULL; i++) {  
        temp = temp->next;  
    }  
  
    if(temp == NULL) {  
        printf("Position out of range!\n");  
        free(newNode);  
        return;  
    }  
  
    newNode->next = temp->next;  
    temp->next = newNode;  
}
```

```
int main() {  
    struct Node* head = NULL;  
  
    // Insert at end  
    insertEnd(&head, 10);  
    insertEnd(&head, 20);  
    insertEnd(&head, 30);  
    display(head);  
  
    // Insert at beginning  
    insertBeginning(&head, 5);  
    display(head);  
  
    // Insert at specific position  
    insertAtPosition(&head, 15, 3); // insert 15 at position 3  
    display(head);  
  
    return 0;  
}
```

Output

Linked List: 10 -> 20 -> 30 -> NULL

Linked List: 5 -> 10 -> 20 -> 30 -> NULL

Linked List: 5 -> 10 -> 15 -> 20 -> 30 -> NULL

12 ? Write a program to delete a node from a singly linked list for all three conditions: (i) Deletion of the first node (beginning) (ii) Deletion of the last node (end) (iii) Deletion of a node from a specific position

Aim

To write a C program that deletes a node from a **singly linked list** in three cases:

1. Deletion of the first node (beginning)
2. Deletion of the last node (end)
3. Deletion of a node from a specific position

Algorithm

1. Define a struct Node with data and next.
2. Create functions for deletion:
 - **Delete at beginning:**
 - Point head to the second node.
 - Free the first node.
 - **Delete at end:**
 - Traverse to the second-last node.
 - Set its next = NULL.
 - Free the last node.
 - **Delete at specific position:**
 - Traverse to the node just before the desired position.
 - Adjust pointers to skip the node at that position.
 - Free the node.
3. Create a function to display the linked list.
4. In main(), demonstrate all three deletion cases.

Code

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Define node structure
```

```
struct Node {
```

```
    int data;
```

```
    struct Node* next;
```

```
};
```

```
// Function to display linked list
```

```
void display(struct Node* head) {
```

```
    struct Node* temp = head;
```

```
    printf("Linked List: ");
```

```
    while(temp != NULL) {
```

```
        printf("%d -> ", temp->data);
```

```
        temp = temp->next;
```

```
    }
```

```
    printf("NULL\n");
```

```
}
```

```
// Insert at end (helper function)
```

```
void insertEnd(struct Node** head, int value) {
```

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
    newNode->data = value;
```

```
    newNode->next = NULL;
```

```
if(*head == NULL) {  
    *head = newNode;  
    return;  
}
```

```
struct Node* temp = *head;  
while(temp->next != NULL) {  
    temp = temp->next;  
}  
temp->next = newNode;  
}
```

// Delete at beginning

```
void deleteBeginning(struct Node** head) {  
    if(*head == NULL) {  
        printf("List is empty!\n");  
        return;  
    }  
    struct Node* temp = *head;  
    *head = (*head)->next;  
    free(temp);  
}
```

// Delete at end

```

void deleteEnd(struct Node** head) {
    if(*head == NULL) {
        printf("List is empty!\n");
        return;
    }
    if((*head)->next == NULL) {
        free(*head);
        *head = NULL;
        return;
    }
    struct Node* temp = *head;
    while(temp->next->next != NULL) {
        temp = temp->next;
    }
    free(temp->next);
    temp->next = NULL;
}

```

// Delete at specific position

```

void deleteAtPosition(struct Node** head, int pos) {
    if(*head == NULL) {
        printf("List is empty!\n");
        return;
    }
    struct Node* temp = *head;

```



```
if(pos == 1) {
```

```
    *head = temp->next;
```

```
    free(temp);
```

```
    return;
```

```
}
```

```
for(int i = 1; temp != NULL && i < pos-1; i++) {
```

```
    temp = temp->next;
```

```
}
```

```
if(temp == NULL || temp->next == NULL) {
```

```
    printf("Position out of range!\n");
```

```
    return;
```

```
}
```

```
struct Node* delNode = temp->next;
```

```
temp->next = delNode->next;
```

```
free(delNode);
```

```
}
```

```
int main() {
```

```
    struct Node* head = NULL;
```

```
    // Create list
```

```
insertEnd(&head, 10);
```

```
insertEnd(&head, 20);
```

```
insertEnd(&head, 30);
```

```
insertEnd(&head, 40);
```

```
insertEnd(&head, 50);
```

```
display(head);
```

```
// Delete at beginning
```

```
deleteBeginning(&head);
```

```
display(head);
```

```
// Delete at end
```

```
deleteEnd(&head);
```

```
display(head);
```

```
// Delete at specific position
```

```
deleteAtPosition(&head, 2); // delete node at position 2
```

```
display(head);
```

```
return 0;
```

```
}
```

Output

Linked List: 10 -> 20 -> 30 -> 40 -> 50 -> NULL

Linked List: 20 -> 30 -> 40 -> 50 -> NULL

Linked List: 20 -> 30 -> 40 -> NULL

Linked List: 20 -> 40 -> NULL

13 ☐ Write a program to implement a Binary Tree.

Aim

To write a C program that implements a **Binary Tree** with basic operations such as:

1. Creating nodes
2. Inserting nodes
3. Traversing the tree (Inorder, Preorder, Postorder)

Algorithm

1. Define a struct Node with:
 - data (to store the value)
 - left (pointer to left child)
 - right (pointer to right child).
2. Create a function createNode() to allocate memory for a new node.
3. Create an insert() function to insert nodes into the tree (using Binary Search Tree logic for simplicity).
4. Implement traversal functions:
 - **Inorder Traversal** (Left → Root → Right)
 - **Preorder Traversal** (Root → Left → Right)
 - **Postorder Traversal** (Left → Right → Root)
5. In main(), build a sample tree, then display traversals.

Code

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Define node structure
```

```
struct Node {  
    int data;  
    struct Node* left;  
    struct Node* right;  
};
```

```
// Function to create a new node
```

```
struct Node* createNode(int value) {  
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));  
    newNode->data = value;  
    newNode->left = newNode->right = NULL;  
    return newNode;  
}
```

```
// Function to insert node (BST logic)
```

```
struct Node* insert(struct Node* root, int value) {  
    if(root == NULL) {  
        return createNode(value);  
    }  
    if(value < root->data) {  
        root->left = insert(root->left, value);  
    } else {  
        root->right = insert(root->right, value);  
    }  
}
```

```
    return root;
}
```

// Inorder Traversal (Left → Root → Right)

```
void inorder(struct Node* root) {
    if(root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}
```

// Preorder Traversal (Root → Left → Right)

```
void preorder(struct Node* root) {
    if(root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}
```

// Postorder Traversal (Left → Right → Root)

```
void postorder(struct Node* root) {
    if(root != NULL) {
        postorder(root->left);
```

```
        postorder(root->right);
        printf("%d ", root->data);
    }
}
```

```
int main() {
    struct Node* root = NULL;

    // Insert nodes into the tree
    root = insert(root, 50);
    insert(root, 30);
    insert(root, 70);
    insert(root, 20);
    insert(root, 40);
    insert(root, 60);
    insert(root, 80);

    printf("Inorder Traversal: ");
    inorder(root);
    printf("\n");

    printf("Preorder Traversal: ");
    preorder(root);
    printf("\n");
}
```

```
printf("Postorder Traversal: ");  
postorder(root);  
printf("\n");  
  
return 0;  
}
```

Output

Inorder Traversal: 20 30 40 50 60 70 80

Preorder Traversal: 50 30 20 40 70 60 80

Postorder Traversal: 20 40 30 60 80 70 50